Lab 2: Agile Backlog Creation & Sprint Simulation

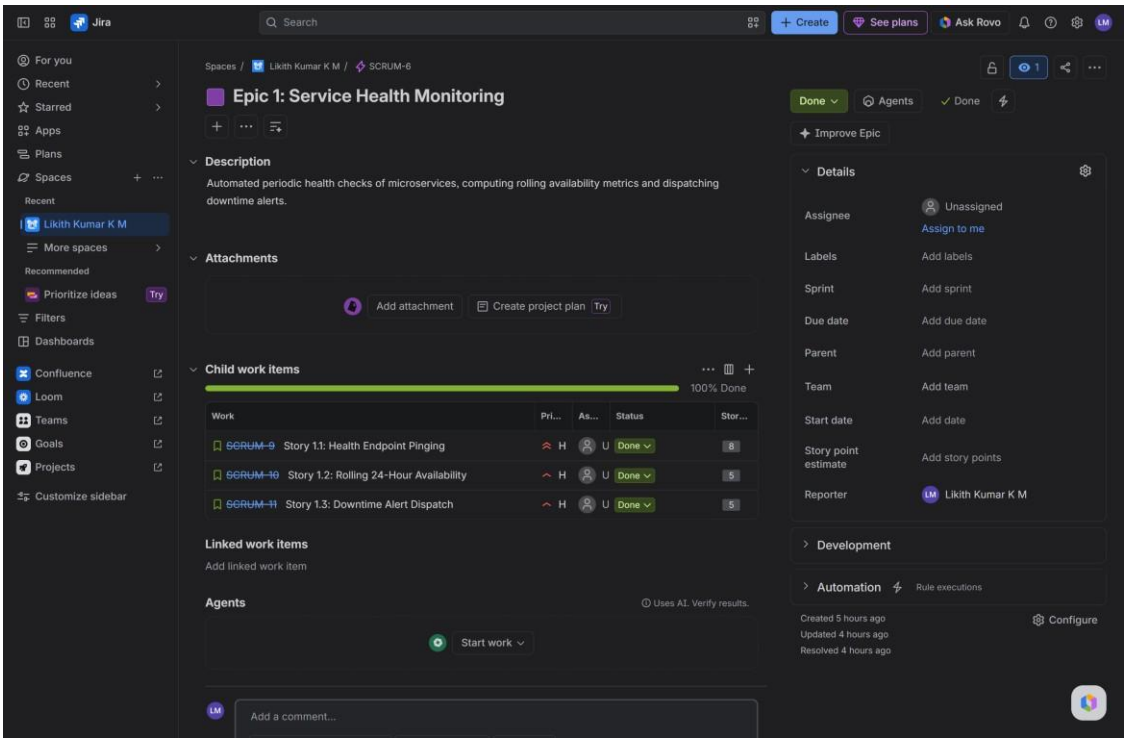
Deliverables — Internal Microservice Catalog & Health Portal

Likith Kumar K M | Jira Project: SCRUM

PES1UG24AM373

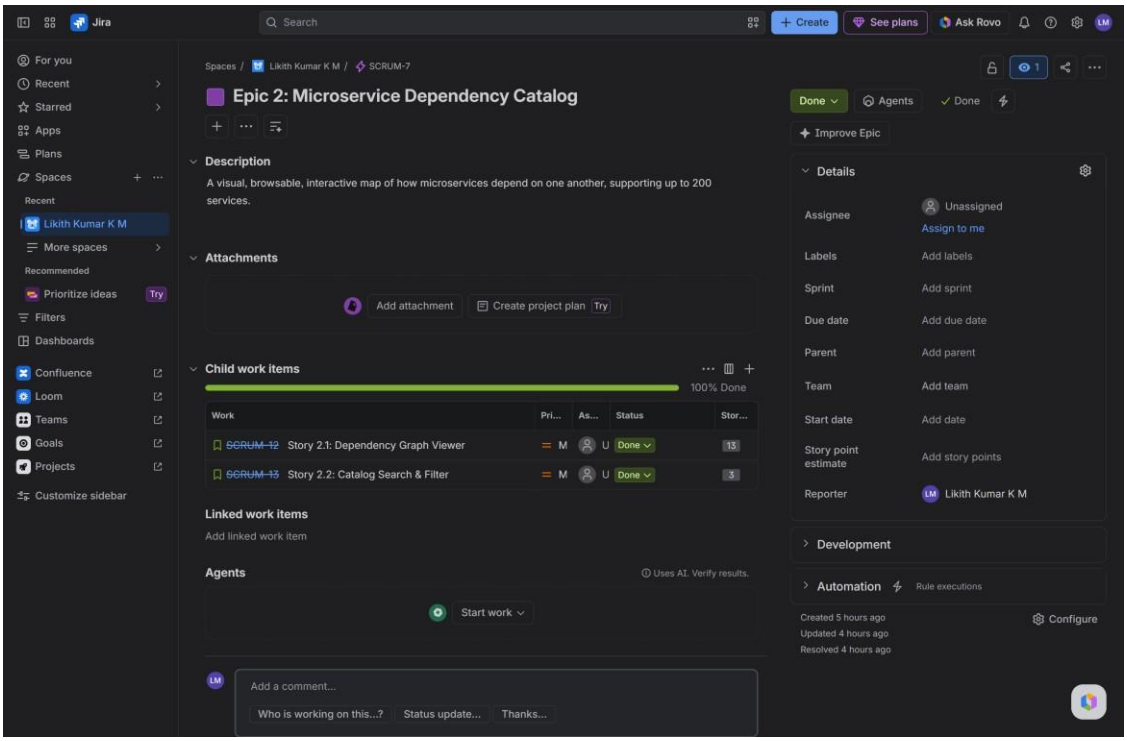
1. Backlog: Epics, User Stories & Story Point Assignments

Epic 1: Service Health Monitoring



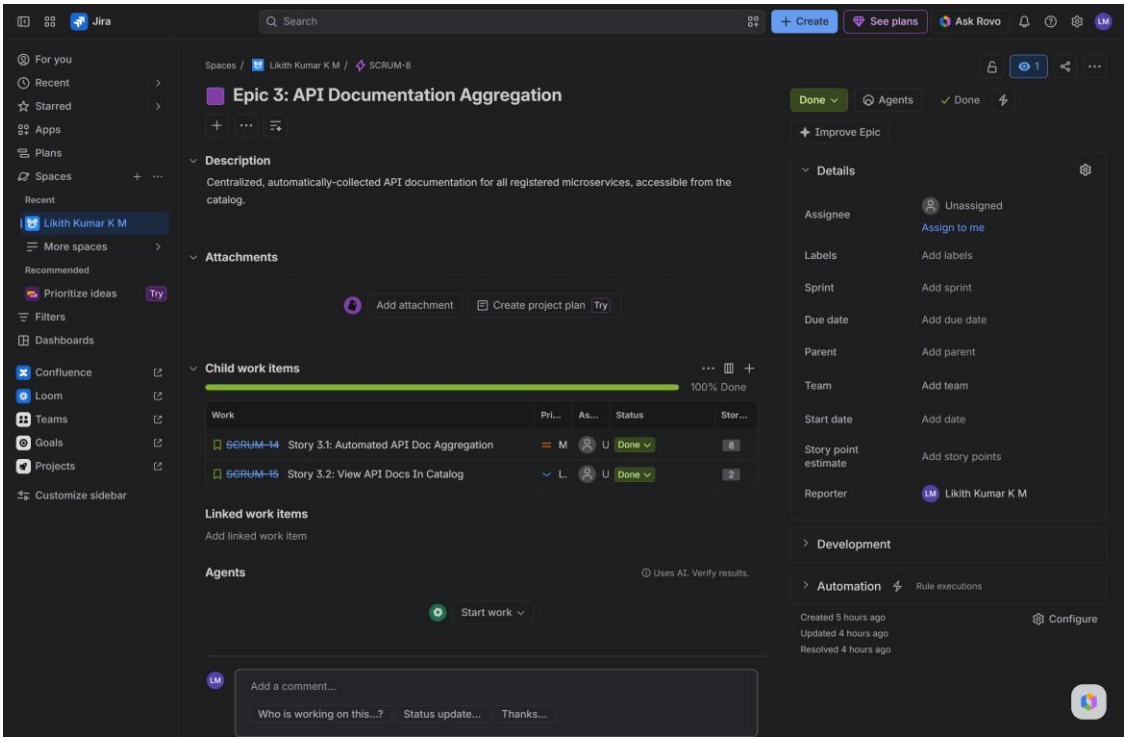
Stories 1.1–1.3 — all Done, story points 8 / 5 / 5

Epic 2: Microservice Dependency Catalog



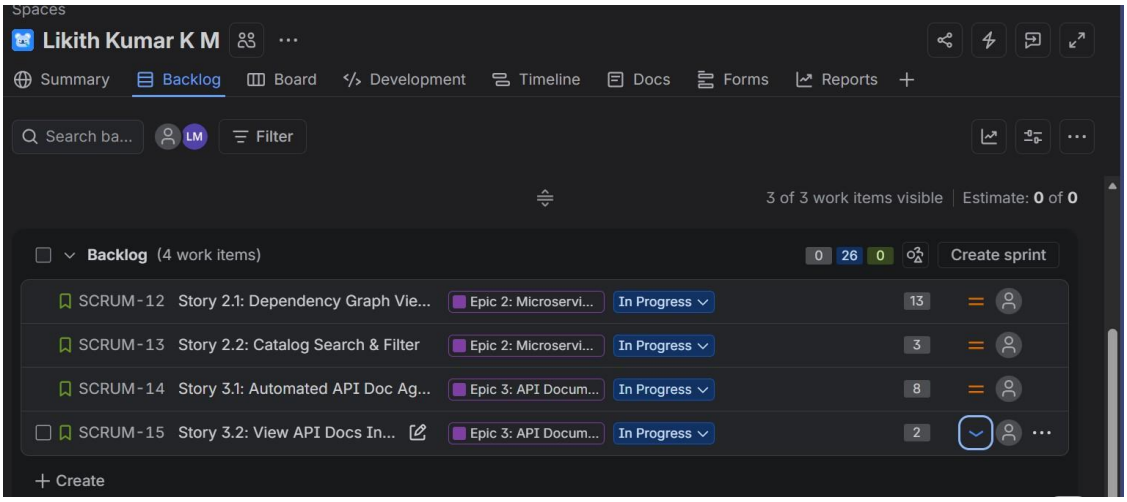
Stories 2.1–2.2 — all Done, story points 13 / 3

Epic 3: API Documentation Aggregation



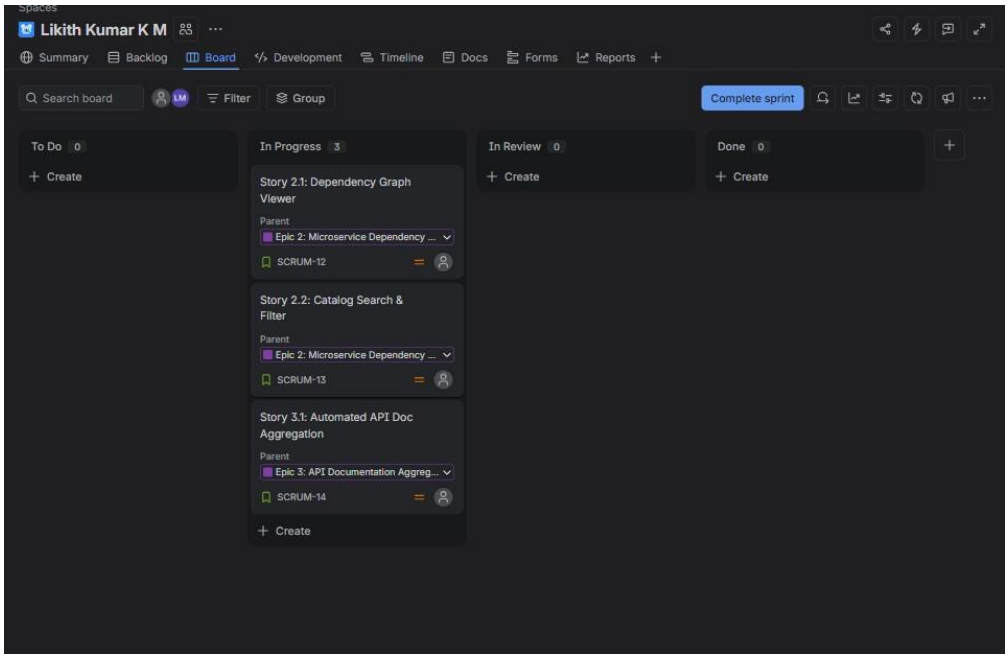
Stories 3.1–3.2 — all Done, story points 8 / 2

Sprint 2 Backlog



Sprint 2 backlog — Stories 2.1, 2.2, 3.1, 3.2 in progress, 26 story points estimated **Sprint**

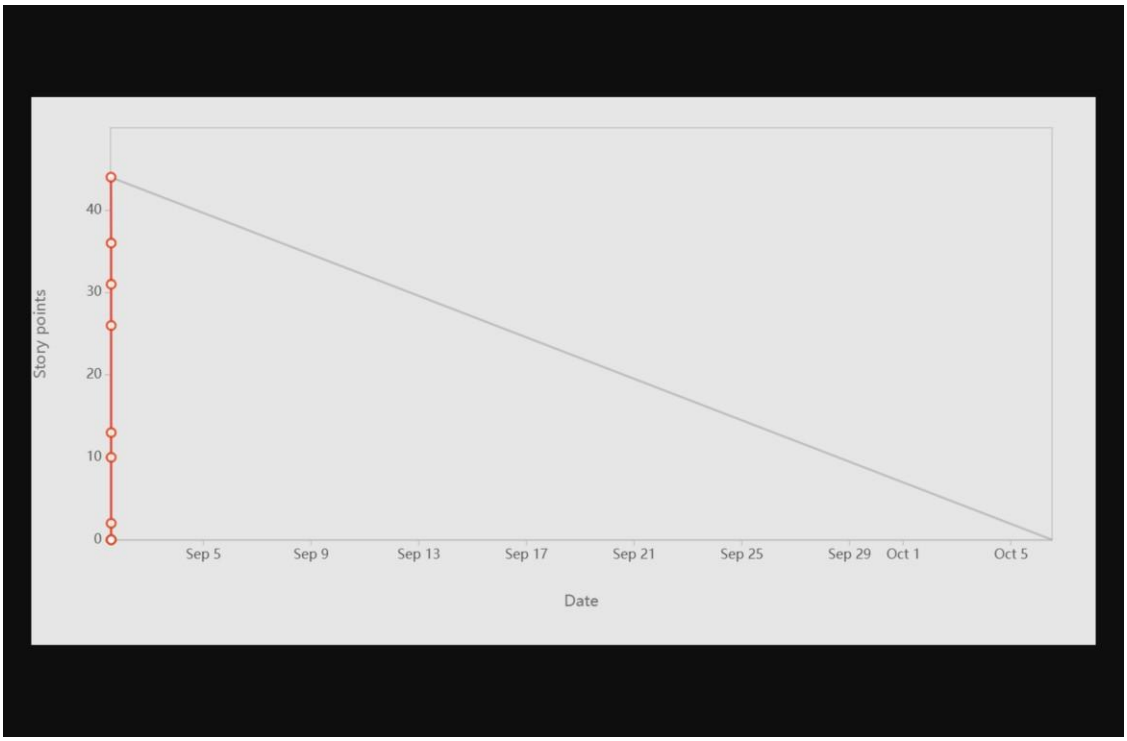
Board



Sprint board — Stories 2.1, 2.2, 3.1 In Progress

2. Sprint Burndown Chart

Story points burned down from 44 to 0 across the sprint, in six steps corresponding to stories being completed at different points rather than all at once — consistent with the simulated sprint progression (To Do → In Progress → Done).



Sprint burndown chart — 44 to 0 story points

Project Summary

I built this backlog directly in Jira, starting from the Lab 1 problem statement for the Internal Microservice Catalog & Health Portal. It's organized into three Epics: automated health monitoring, the dependency catalog, and API documentation aggregation. Under these I wrote seven user stories in the “As a / I want / So that” format, estimated them with the Fibonacci scale, and ran them through two one-week sprints.

Epic	Story	Priority	Points	Status
Epic 1: Service Health Monitoring	1.1 Health Endpoint Pinging	Highest	8	Done
	1.2 Rolling 24-Hour Availability	High	5	Done
	1.3 Downtime Alert Dispatch	High	5	Done
Epic 2: Microservice Dependency Catalog	2.1 Dependency Graph Viewer	Medium	13	Done
	2.2 Catalog Search & Filter	Medium	3	Done
Epic 3: API Documentation Aggregation	3.1 Automated API Doc Aggregation	Medium	8	Done
	3.2 View API Docs In Catalog	Low	2	Done

Reflection Questions

1. Did your estimations reflect the actual effort?

For the most part. The 13-point Dependency Graph Viewer and the two 8-point stories (Health Endpoint Pinging, API Doc Aggregation) were the ones with real technical uncertainty, things like rendering a graph at scale or getting the scheduler and integration logic right, so it made sense for Planning Poker to rank them higher. The 2 and 3 point stories really were simpler UI and filtering work, so that part checked out too. That said, this was a simulated sprint, not actual implementation, so none of these estimates got tested against a real blocker. What I have is our best guess at relative complexity going in, not effort that was actually verified by delivery.

2. Was your backlog well-prioritized?

I think so. Priority followed the dependency chain laid out in the problem statement. Health monitoring (Epic 1) got Highest/High because it's the core value proposition in FR-001 and it's what delivers the alerting the whole portal is supposed to provide. The Dependency Catalog and API Documentation epics were Medium since they matter but aren't as time-critical, and the smallest, most cosmetic story (viewing docs inline) got Low. Because of this, the riskiest and highest-value work landed in Sprint 1 instead of getting pushed to the end.

3. How did your simulated sprint align with your plan?

Both sprints finished everything that was planned, with no carryover. Sprint 1 wrapped up the three Service Health Monitoring stories, and Sprint 2 closed out the Dependency Catalog and API Documentation stories. Since this was a simulation and not real development, there weren't any unplanned blockers or scope changes along the way, so the sprint matched the plan exactly. I'd call that a limitation of the exercise more than a sign everything went perfectly (more on this in Q4).

4. What insights did the burndown chart give about your team's capacity?

Because every story finished on schedule with no scope creep mid-sprint, the burndown line stays close to the ideal guideline instead of showing the sharp drops or flat stretches that usually point to bad estimation or blocked work. In an actual sprint, though, a burndown this clean would be a little suspicious. It can mean the team had slack and under-committed rather than that the estimates were flawless. So the real takeaway here is procedural: the shape of a burndown line is a diagnostic tool. Sharp late drops mean scope creep, a flat line that crashes at the end means overcommitment, and finishing well under the guideline early means undercommitment. These are patterns worth watching for once this backlog moves into actual implementation.